

Tarea Programada 1

MiniOS: Simulador básico de procesos

Fecha de Entrega: 7 de Octubre de 2026 antes de la media noche.

1. Marco Contextual

Un sistema operativo debe administrar recursos limitados y decidir cómo utilizarlos cuando existen múltiples tareas que desean ejecutarse. Entre estos recursos se encuentran el procesador, la memoria, los dispositivos de entrada y salida, el almacenamiento y otros recursos compartidos.

Uno de los problemas fundamentales de un sistema operativo consiste en decidir qué tarea utilizará el procesador en determinado momento.

En esta primera tarea se desarrollará una simulación simplificada de este problema, con un un modelo simplificado que permita posteriormente estudiar conceptos como

- procesos;
- estados de los procesos;
- colas;
- planificación del procesador;
- utilización del CPU;
- tiempos de espera;
- concurrencia;
- memoria;
- entrada y salida;
- rendimiento del sistema.

Este simulador podrá ser ampliado posteriormente durante el curso.

2. Objetivo general

Desarrollar en Python un simulador básico de ejecución de procesos que permita representar la llegada, espera, ejecución y finalización de diferentes trabajos utilizando un único procesador.

El programa representará un computador con **un único procesador**, al cual llegan diferentes procesos que requieren determinada cantidad de tiempo para completar su ejecución.

En esta primera versión se utilizará una política sencilla:

- Los procesos serán atendidos en el mismo orden en que llegan al sistema.
- Este comportamiento corresponde conceptualmente a una cola FIFO (*First In, First Out*).

3. Objetivos específicos

Al finalizar la tarea, el estudiante deberá ser capaz de:

1. representar procesos mediante estructuras de datos;
2. modelar el paso del tiempo dentro de una simulación;
3. implementar una cola FIFO;
4. comprender la diferencia entre llegada, espera, ejecución y finalización de un proceso;
5. calcular métricas básicas relacionadas con la ejecución de procesos;
6. leer información desde archivos de texto estructurados;
7. generar resultados en archivos para su posterior análisis;
8. interpretar el comportamiento de un sistema cuando varios procesos compiten por un recurso limitado.
9. Debe desarrollar un programa llamado: **minios.py**
10. El programa deberá simular el comportamiento de un computador que posee:
 - un único CPU;
 - una cola de procesos esperando;
 - procesos que llegan en diferentes momentos;
 - ejecución no interrumpida del proceso actual.

Una vez que un proceso empieza a utilizar el CPU, deberá continuar ejecutándose hasta terminar.

4. Escenario

Imagine un servidor utilizado para ejecutar tareas relacionadas con procesamiento de datos. Algunos ejemplos podrían ser cargar un conjunto de datos, transformar registros, calcular estadísticas, etc.

Cada una de estas tareas será representada como un **proceso**.

Cada proceso tendrá inicialmente tres características principales:

- un identificador;
- un instante de llegada al sistema;
- una cantidad de tiempo de CPU necesaria para completar su ejecución.

Por ejemplo:

Proceso Llegada Tiempo requerido de CPU

P1	0	5
P2	2	3
P3	3	8
P4	6	2

Esto significa, por ejemplo, que:

- P1 llega al sistema en el instante 0;
- necesita 5 unidades de tiempo de CPU;
- P2 llega en el instante 2;
- si el procesador está ocupado, deberá esperar.

6. Archivo de entrada

Los procesos deberán ser cargados desde un archivo CSV.

El programa deberá trabajar como mínimo con un archivo llamado:
procesos.csv

Ejemplo:

id,llegada,duracion

P1,0,5

P2,2,3

P3,3,8

Significado de las columnas

Columna	Significado	Ejemplo
Id	Identificador único del proceso.	P1 P2 P3 Limpieza01 ModeloA
llegada	Unidad de tiempo en la cual el proceso ingresa al sistema.	Debe ser un número entero mayor o igual que cero.
duracion	Cantidad de unidades de tiempo de CPU que necesita el proceso para terminar.	Debe ser un número entero mayor que cero.

7. Restricciones sobre los datos

El programa deberá validar al menos las siguientes situaciones:

- identificadores vacíos;
- identificadores duplicados;
- tiempos de llegada negativos;
- duraciones iguales o menores que cero;
- valores que no puedan convertirse a números enteros;
- archivo inexistente;
- archivo vacío;
- filas incompletas.

Cuando se encuentre un error, el programa deberá mostrar un mensaje comprensible.

No es necesario intentar reparar automáticamente el archivo.

8. Ejemplo de funcionamiento

La simulación deberá comenzar en:

tiempo = 0

El tiempo avanzará en unidades enteras:

0, 1, 2, 3, 4, 5...

En cada unidad de tiempo el simulador deberá considerar:

1. cuáles procesos llegan en ese instante;
2. cuáles deben ingresar a la cola;
3. si el CPU está libre;
4. cuál proceso debe comenzar a ejecutarse;
5. cuánto tiempo de ejecución le queda al proceso actual;
6. si algún proceso terminó.

9. Política de ejecución

La primera versión de MiniOS deberá utilizar una cola FIFO.

FIFO significa:

First In, First Out.

El primer proceso que entra a la cola debe ser el primero en salir de ella.

Si varios procesos llegan exactamente en el mismo instante, deberán respetar el orden en que aparecen en el archivo de entrada.

10. Ejemplo conceptual

Considere:

P1 llega en 0 y dura 5

P2 llega en 2 y dura 3

P3 llega en 3 y dura 4

El comportamiento esperado sería aproximadamente:

Tiempo 0: llega P1

Tiempo 0: P1 comienza ejecución

Tiempo 1: P1 continúa ejecutándose

Tiempo 2: llega P2

Tiempo 2: P2 entra a la cola

Tiempo 2: P1 continúa ejecutándose

Tiempo 3: llega P3

Tiempo 3: P3 entra a la cola

Tiempo 3: P1 continúa ejecutándose

Tiempo 4: P1 continúa ejecutándose

Tiempo 5: P1 termina

Tiempo 5: P2 comienza ejecución

La representación exacta de los mensajes queda a criterio del estudiante, siempre que el comportamiento pueda entenderse con claridad.

11. Estados mínimos de los procesos

Para esta primera tarea se utilizarán conceptualmente los siguientes estados:

Nuevo

El proceso todavía no ha llegado al sistema.

Listo

El proceso llegó, pero se encuentra esperando por el procesador.

Ejecutando

El proceso está utilizando el CPU.

Terminado

El proceso completó toda su ejecución.

12. Datos que deberá almacenar cada proceso

Como mínimo, cada proceso deberá manejar:

- identificador
- tiempo de llegada (llegada en el archivo)
- duración (duración en el archivo)
- tiempo de inicio (momento en que empieza a ejecutarse)
- tiempo de finalización (momento en que termina)
- tiempo de espera (cuanto tiempo esperó en cola antes de empezar a ejecutarse)
- tiempo de retorno: ver siguiente sección.

13. Cálculo de métricas

El simulador deberá calcular para cada proceso las siguientes métricas.

Tiempo de retorno

También conocido como *turnaround time*.

Se calcula como:

$\text{tiempo_retorno} = \text{tiempo_finalizacion} - \text{tiempo_llegada}$

Representa cuánto tiempo permaneció el proceso dentro del sistema.

Tiempo de espera

Para esta primera versión, dado que los procesos no serán interrumpidos:

$\text{tiempo_espera} = \text{tiempo_inicio} - \text{tiempo_llegada}$

Representa cuánto tiempo estuvo esperando antes de obtener el CPU.

14. Métricas globales

Al terminar la simulación deberán mostrarse al menos:

- cantidad total de procesos;
- tiempo total de simulación; (o sea desde cero hasta que terminó el último proceso en el archivo)
- tiempo promedio de espera;
- tiempo promedio de retorno;
- proceso con mayor tiempo de espera;
- proceso con menor tiempo de espera.

15. Salida en pantalla

Al finalizar la ejecución deberá presentarse un resumen similar al siguiente:

```
=====
      RESULTADO MINIOS
=====
```

Proceso	Llegada	Inicio	Fin	Espera	Retorno
P1	0	0	5	0	5
P2	2	5	8	3	6
P3	3	8	12	5	9

Procesos ejecutados: 3

Tiempo total: 12

Espera promedio: 2.67

Retorno promedio: 6.67

No es necesario utilizar exactamente este formato.

Sin embargo, la información debe presentarse de manera ordenada y comprensible.

16. Archivo de resultados

Además de mostrar resultados en pantalla, el programa deberá generar un archivo: resultados.csv

Con una estructura semejante a:

id,llegada,duracion,inicio,fin,espera,retorno

P1,0,5,0,5,0,5

P2,2,3,5,8,3,6

P3,3,4,8,12,5,9

Este archivo se utilizará posteriormente para análisis de datos.

17. Registro de eventos

El programa deberá generar adicionalmente un archivo:

eventos.txt

En él deberá quedar registrado lo ocurrido durante la simulación.

Por ejemplo:

[0] P1 llega al sistema

[0] P1 inicia ejecución

[2] P2 llega al sistema

[2] P2 entra a la cola

[3] P3 llega al sistema

[3] P3 entra a la cola

[5] P1 termina

[5] P2 inicia ejecución

[8] P2 termina

[8] P3 inicia ejecución

[12] P3 termina

Este archivo funcionará como una especie de registro o *log* del simulador.

18. Restricción tecnológica

El programa deberá desarrollarse utilizando exclusivamente **Python y su biblioteca estándar**.

Por lo tanto, no se permite utilizar bibliotecas externas. NADA QUE REQUIERA PIP INSTALL.

No se permite, por ejemplo:

pandas numpy matplotlib simpy polars scipy

Sí pueden utilizarse módulos pertenecientes a la biblioteca estándar, tales como: csv, collections, statistics, dataclasses, pathlib, random, sys, argparse. Úselas a discreción.

El código deberá separarse en funciones.

No se aceptará como solución adecuada un programa completo escrito únicamente dentro de un bloque secuencial.

Se recomienda contar con funciones semejantes a:

`leer_procesos(...)`

`validar_procesos(...)`

`simular(...)`

`calcular_metricas(...)`

`guardar_resultados(...)`

`guardar_eventos(...)`

`mostrar_resumen(...)`

Los nombres anteriores son solamente ejemplos.

El estudiante puede definir su propia organización.